

Computing Scores in a Complete Search System

Academic Year 2025/2026

Luigi Santangelo
luigi.santangelo@unipv.it

University of Pavia
**Master Degree in
Computer Engineering**

Computing Scores

In a Information Retrieval System each document in the collection is represented by a vector.

The system computes the distance between the vector representing the Query and each Document, then shows K top documents ordered by **decreasing score**

The Cosine Score Algorithm

COSINESCORE(q)

```
1 float Scores[N] = 0
2 Initialize Length[N]
3 for each query term  $t$ 
4 do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5   for each pair( $d, tf_{t,d}$ ) in postings list
6   do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7 Read the array Length[d]
8 for each  $d$ 
9 do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

N = number of documents

Score[N] = vector for document scores (initialized to 0)

Length[N] = vector for document length

$$Length[i] = \sqrt{\sum_{j=1}^M \vec{V}_j(d_i)^2}$$

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

This process of adding in contributions one query term at a time is sometimes known as ***term-at-a-time*** scoring

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

For each term in the query, the weight $w_{t,q}$ is computed. Such a value is multiplied (line 6) with the weight of the term in the document.

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

For each term in the query, the posting list is retrieved, then we compute the weight of the term t in the document.

The weight can be either tf or tf-idf

The computed weight is stored in an **accumulator**

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Scores are normalized

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

K documents are returned

The Cosine Score Algorithm

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

$$sim(d_1, d_2) = \cos\theta =$$

$$\frac{\vec{V}(d_1) \cdot \vec{V}(d_2)}{\|\vec{V}(d_1)\| \|\vec{V}(d_2)\|}$$

$$\sum_{i=1}^M \vec{V}_i(d_1) \vec{V}_i(d_2)$$

This is an invariant because it is the same for all documents (being d_2 here the query)

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Instead of computing the length of the vector representing a document each time the CosineScore function is called, we can precompute this vector only once, during indexing time, because such value is independent to the query

$$Length[i] = \sqrt{\sum_{j=1}^M \vec{V}_j(d_i)^2}$$

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \leftarrow w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

To increase execution time, we avoid computing the weight each time.

Instead, we can store idf_t in the head of the postings for t . The value $tf_{t,d}$ instead is stored on each posting entry.

The Cosine Score Algorithm: increasing performance

COSINESCORE(q)

```
1  float Scores[N] = 0
2  Initialize Length[N]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[d] +=  $wf_{t,d} \times w_{t,q}$ 
7  Read the array Length[d]
8  for each  $d$ 
9  do Scores[d] = Scores[d] / Length[d]
10 return Top K components of Scores[]
```

Sorting all documents
depending on the scores can
take much time.

An alternative is to use an Max
Heap

The Cosine Score Algorithm: the big problem!

COSINESCORE(q)

```
1  float  $Scores[N] = 0$ 
2  Initialize  $Length[N]$ 
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5    for each pair( $d, tf_{t,d}$ ) in postings list
6    do  $Scores[d] += w_{t,q} \times tf_{t,d}$ 
7  Read the array  $Length[d]$ 
8  for each  $d$ 
9  do  $Scores[d] = Scores[d] / Length[d]$ 
10 return Top  $K$  components of  $Scores[]$ 
```

For each term in the query, the corresponding posting list is retrieved and for each item the score is computed.

Computing the score for millions of documents takes a huge amount of time.

Increasing Performance

We would like to have a method for retrieving K documents that **are likely to be** among the K highest scoring documents for a query.

The idea is to eliminate all those documents from the collection that probably are not interesting.

To identify those documents to be eliminated, some **heuristics** are followed

Increasing Performance

Heuristic 1: We only consider documents containing terms whose *idf* exceeds a preset threshold. The idf value is stored in the **head of the Postings List**. This value is lower when the number of documents where the term appears is large. Low-idf terms are treated as **stop words** and do not contribute to scoring.

Increasing Performance

Heuristic 2: We only consider documents **that contain all (or many)** of the query terms. A danger of this scheme is that by requiring all query terms to be present in a document, we may end up with **fewer than K candidate** documents in the output

Increasing Performance

Heuristic 3: for each term t in the dictionary, we precompute a **list of r documents** having the highest weight (for example tf_idf) for that term t . This list is called **champion list**

We build a set A , given as the union of all champion lists for all terms.

Given a query q , we compute the score between the query q and the documents in A

Of course **r must be much greater than K** . One issue here is that the value r is set at **the time of index construction**, whereas K is application dependent and may not be available until the query is received.

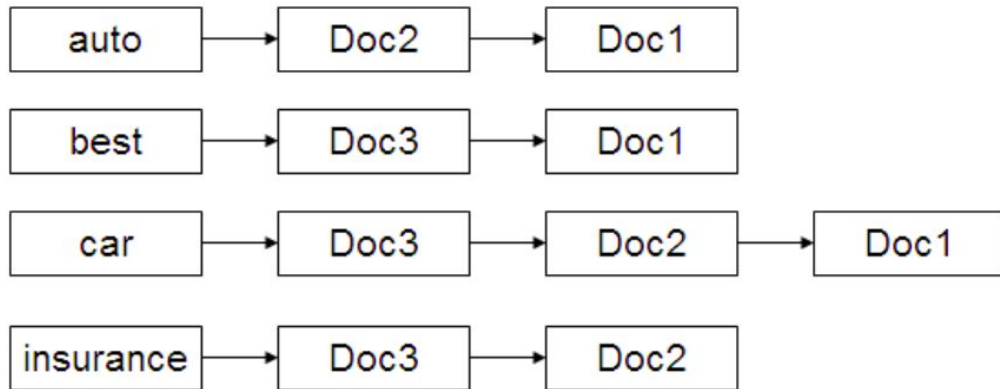
Increasing Performance

Heuristic 4: we assign a **static quality score** to each document $g(d)$.

Such a score is independent from the query and is usually assigned statically to a document. It is a **value between 0 and 1** and it represents the quality of the document.

We can **sort all Posting Lists**, not depending on the ID of the document but depending on the Static Quality Score:

Increasing Performance



In this example, the Posting Lists are sorted by static quality index:

$$g(\text{Doc1}) = 0.25$$

$$g(\text{Doc2}) = 0.5$$

$$g(\text{Doc3}) = 1$$

Increasing Performance

Similarly to the Heuristic 3, here too, we create a Champion List with the first r documents having the largest static quality score.

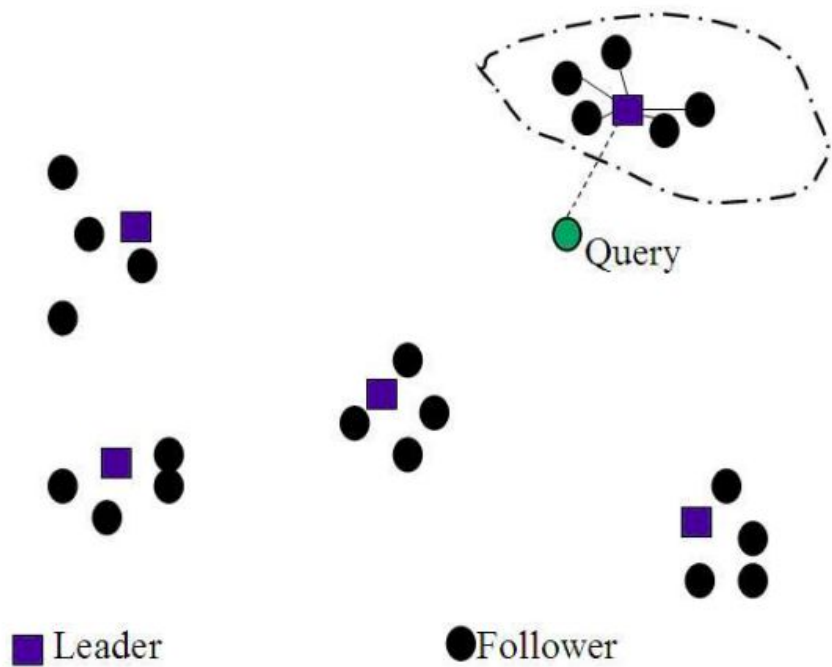
Then at query time, we only compute the score for all the documents belonging to the union of all Champion Lists. The score can be computed as a combination between the quality score and the cosine score

$$\text{net-score}(q, d) = g(d) + \frac{\vec{V}(q) \cdot \vec{V}(d)}{|\vec{V}(q)| |\vec{V}(d)|}$$

Increasing Performance

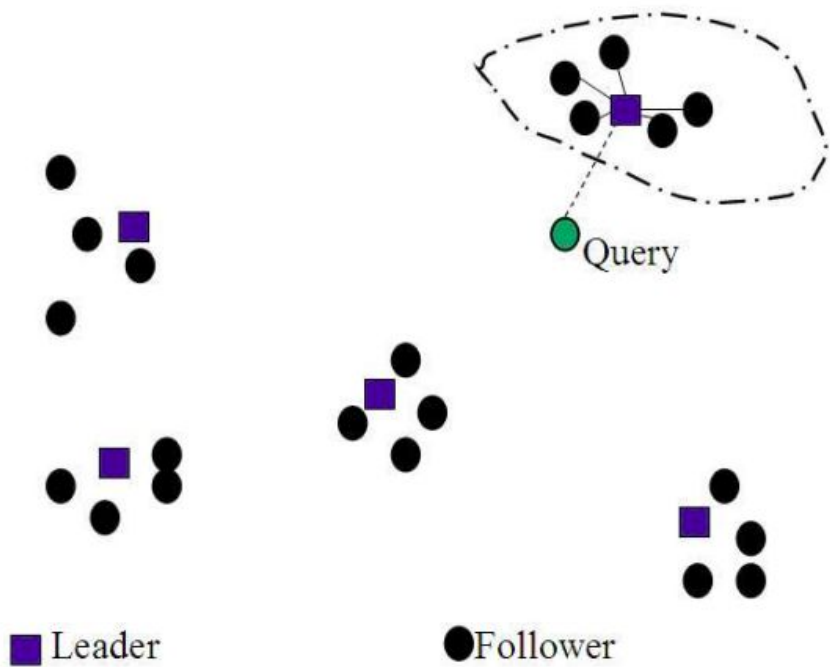
Heuristic 5: we have two preprocessing steps during which document vectors are clustered, then at query time we compute cosine score only with those documents belonging to some clusters as candidate.

This method is called **Cluster Pruning**



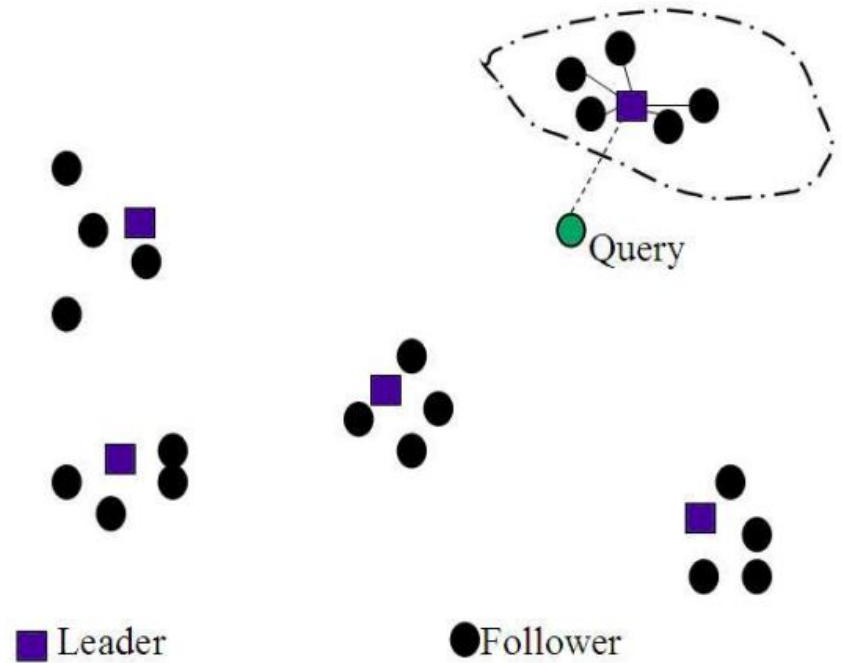
Increasing Performance

1. pick $\sqrt{N}$ documents at random from the collection. Call these **leaders**
2. For each document that is not a leader (called **follower**), we compute its nearest leader (this means computing cosine distance)



Increasing Performance

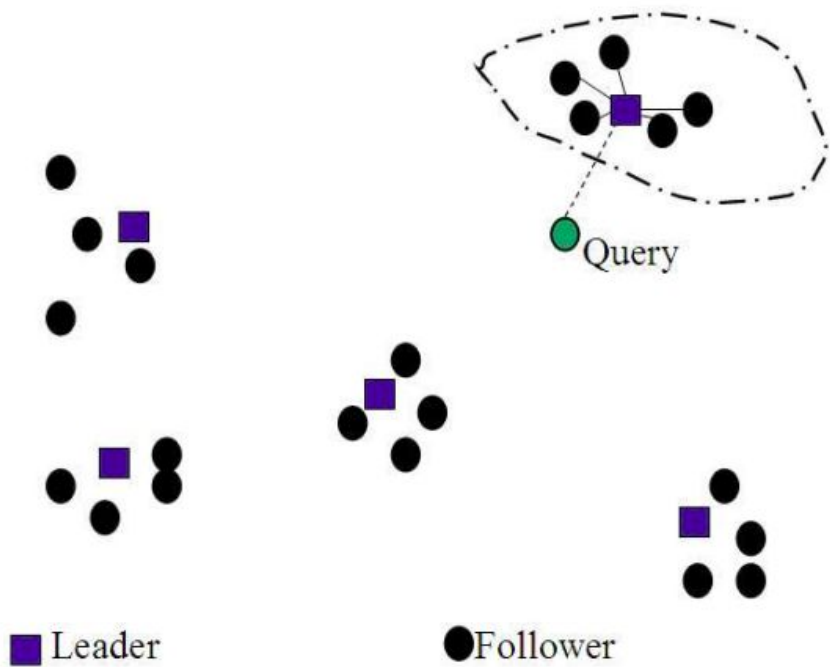
Being the leaders selected randomly,
we expect that number of followers of
each leader is approximately equals to
 $N / \text{sqrt}(N) = \text{sqrt}(N)$



Increasing Performance

Now at query time we can:

1. using the query q , find the leader L which is closest to q . This means that we need to compute the cosine a number of time equals to $\sqrt{N}$
2. build the set A will contain L and its followers
3. compute cosine score for all documents in this candidate set



Drawback of using heuristics

All heuristics aim to reduce the set of documents for which compute the cosine score.

To cut the collection, they all make use of parameter (for example r). Sometimes depending on the terms of the query and the parameters, **the number of documents to the candidate set A is lower than k**

Solution: **Tiered Index**

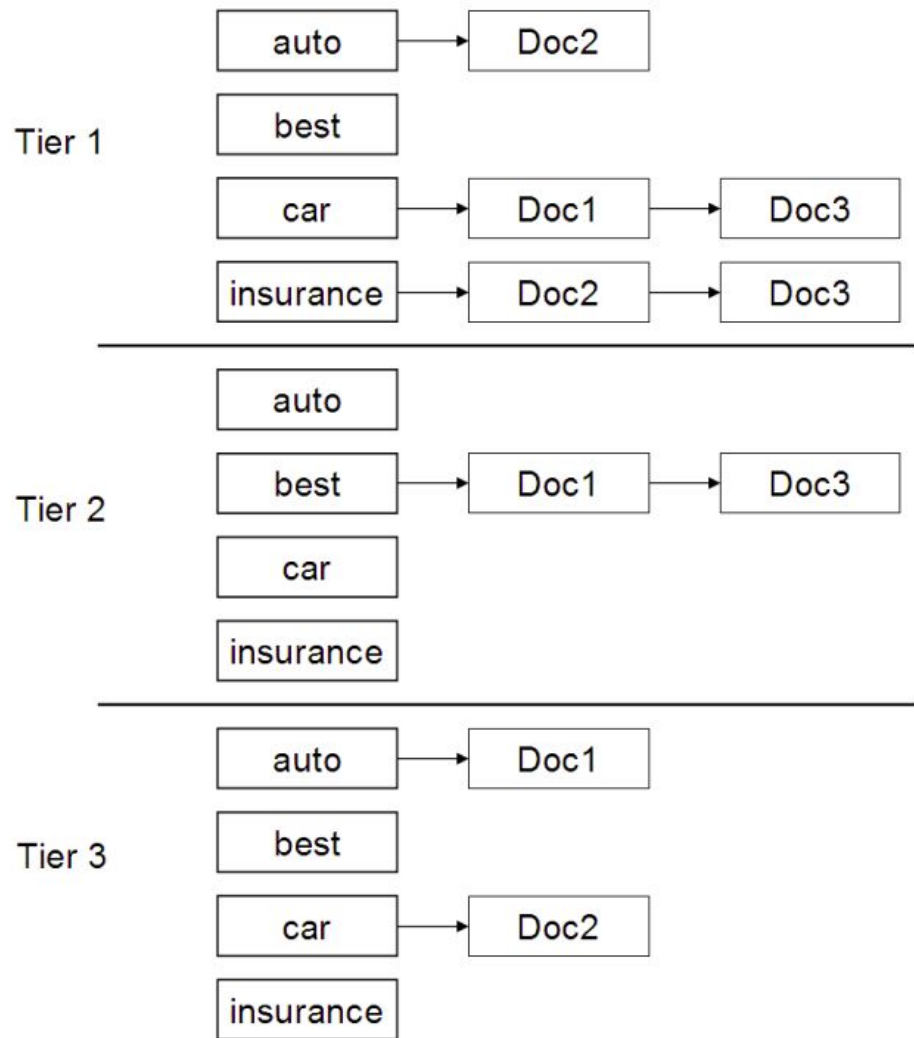
Tiered Index

The picture showed a 3-tiered Index.

Tier 1 indexes all documents having, for example, a `tf_idf` greater 20

Tier 2 indexes all documents having a `tf_idf` between 10 and 20

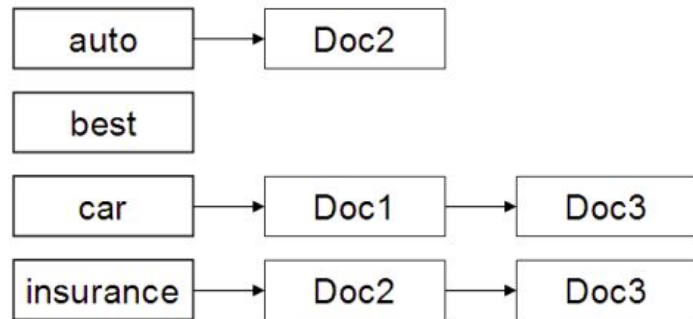
Tier 3 indexes all documents having a `tf_idf` lower than 10



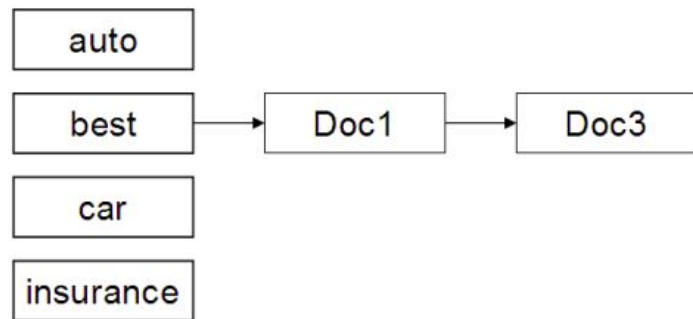
Tiered Index

If we fail to get K results from tier 1, query processing “falls back” to tier 2, and so on

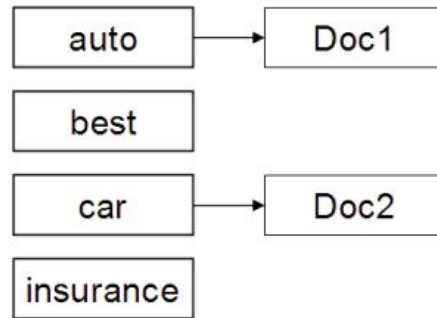
Tier 1



Tier 2

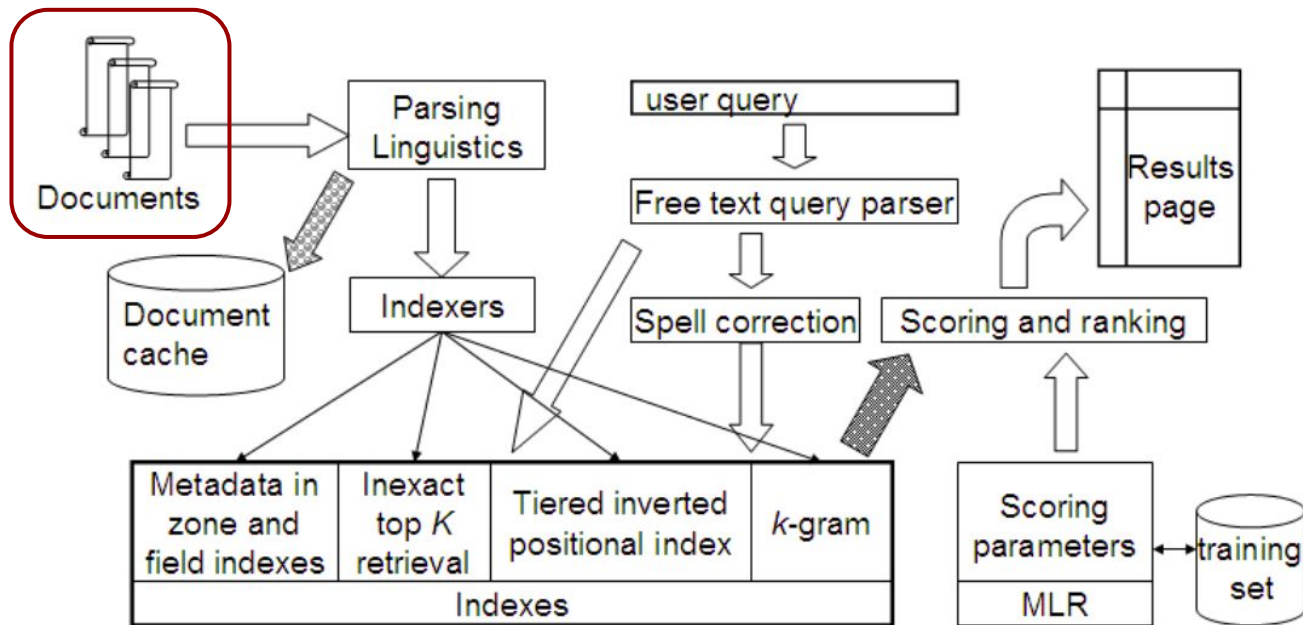


Tier 3



Components of an information retrieval system

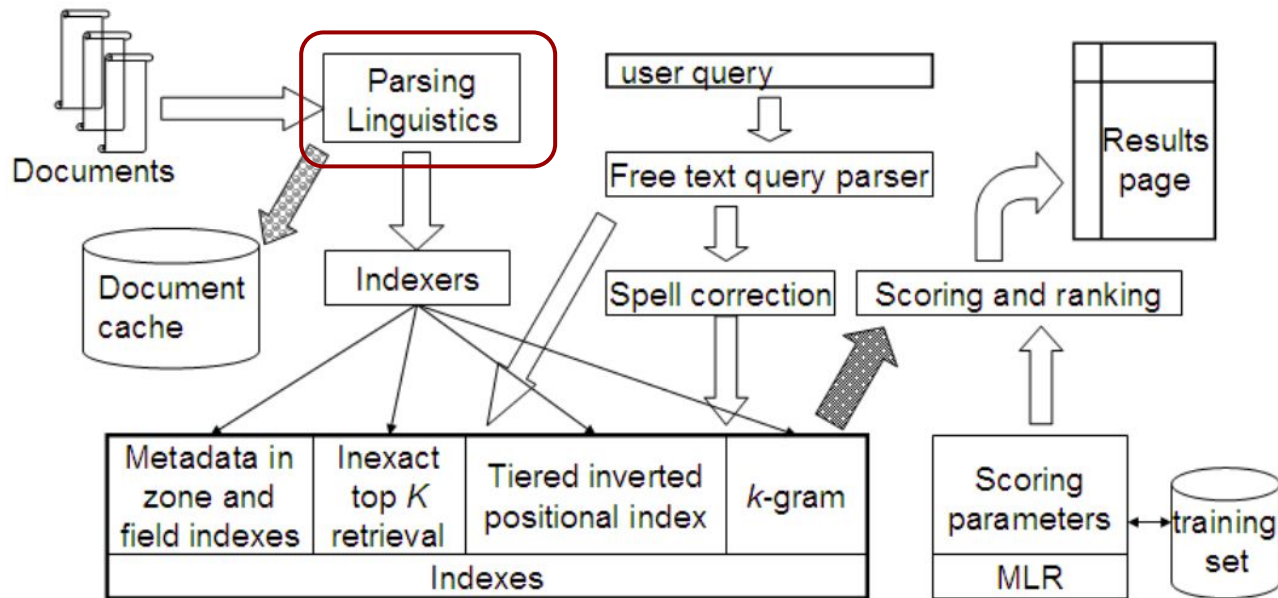
Putting them all together



Collection of documents to be parsed.

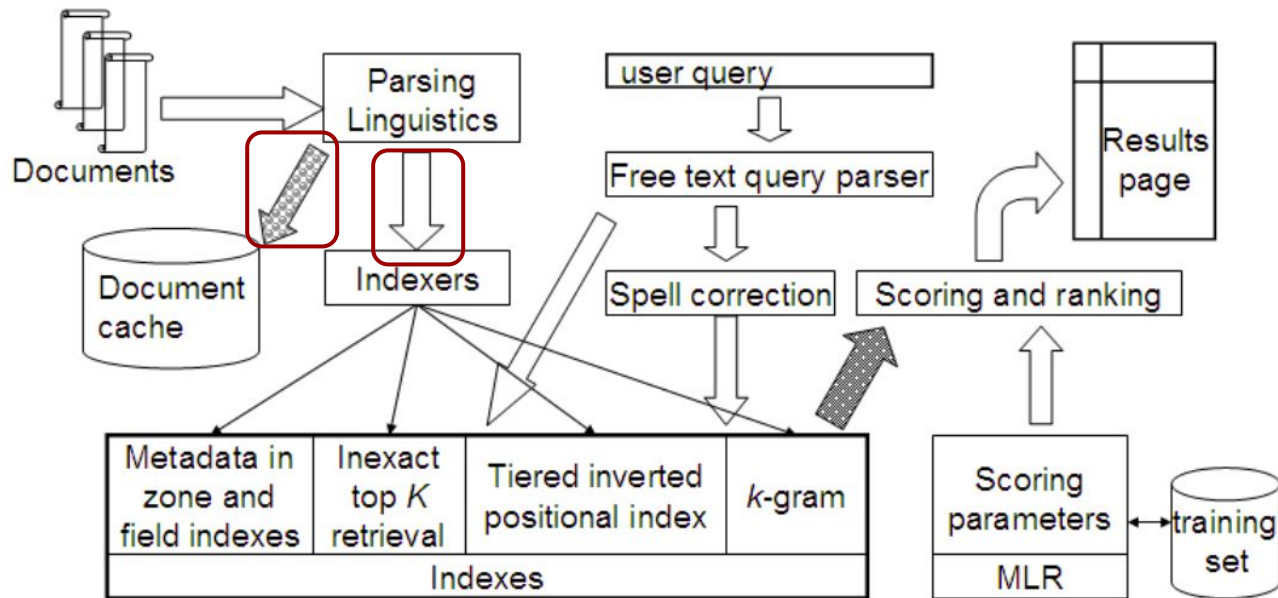
Text is extracted and sent to the Linguistic Preprocessor

Putting them all together



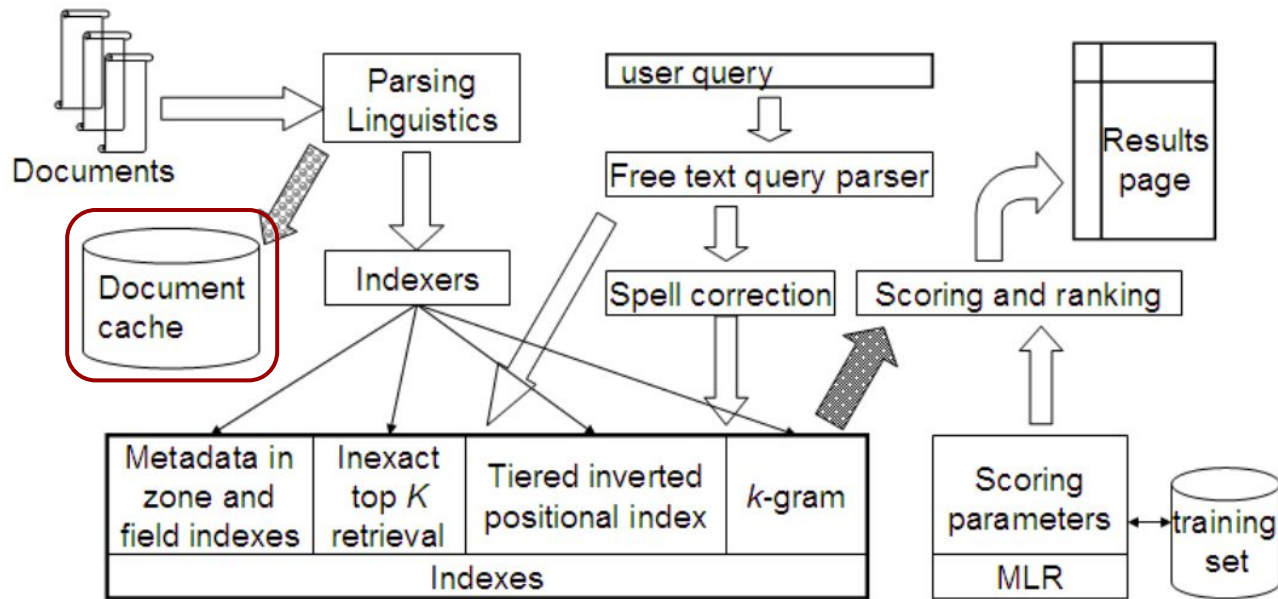
1. detect format
2. detect language
3. tokenization
4. stemming

Putting them all together



All tokens are sent to the indexers and to the document cache

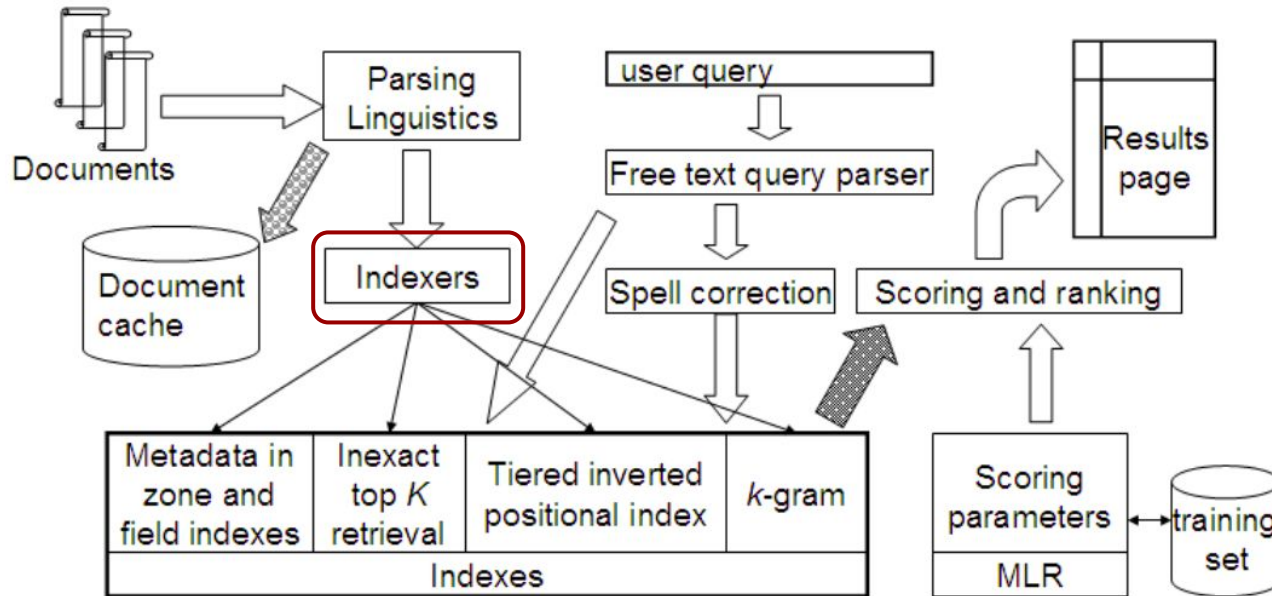
Putting them all together



The Document Cache keeps the parsed document. This is used to create **snippets** of text in the query results.

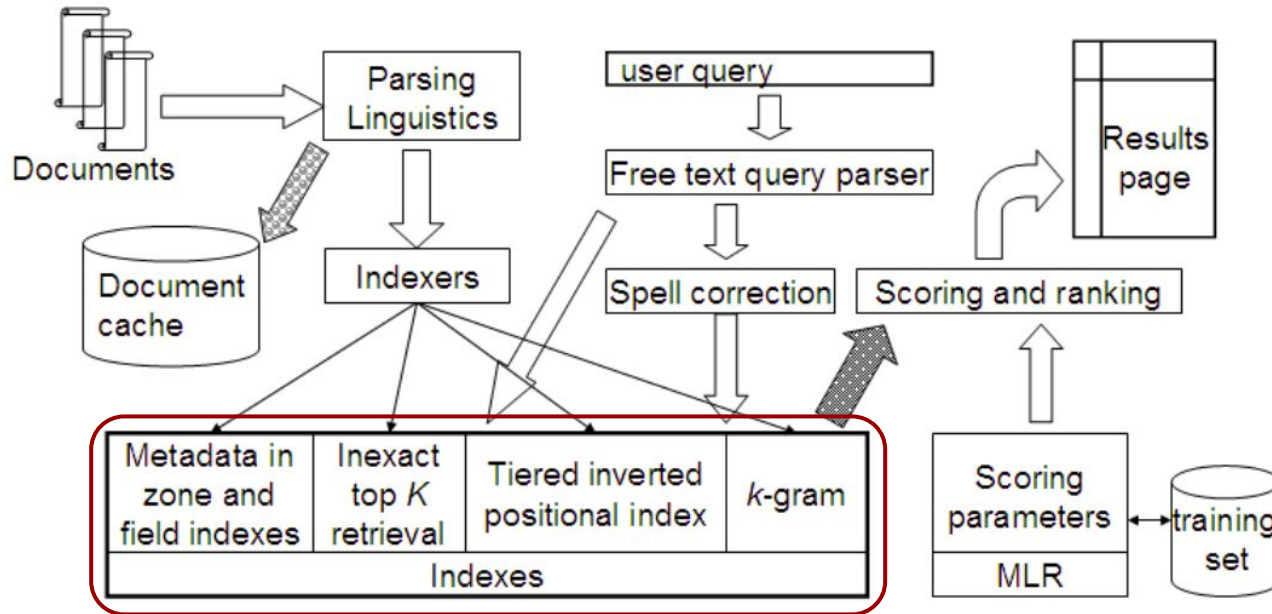
Snippet tries to give an explanation to the user of why the document matches the query

Putting them all together



Tokens are turned into Terms and are stored into the vocabulary. Posting Lists are created.

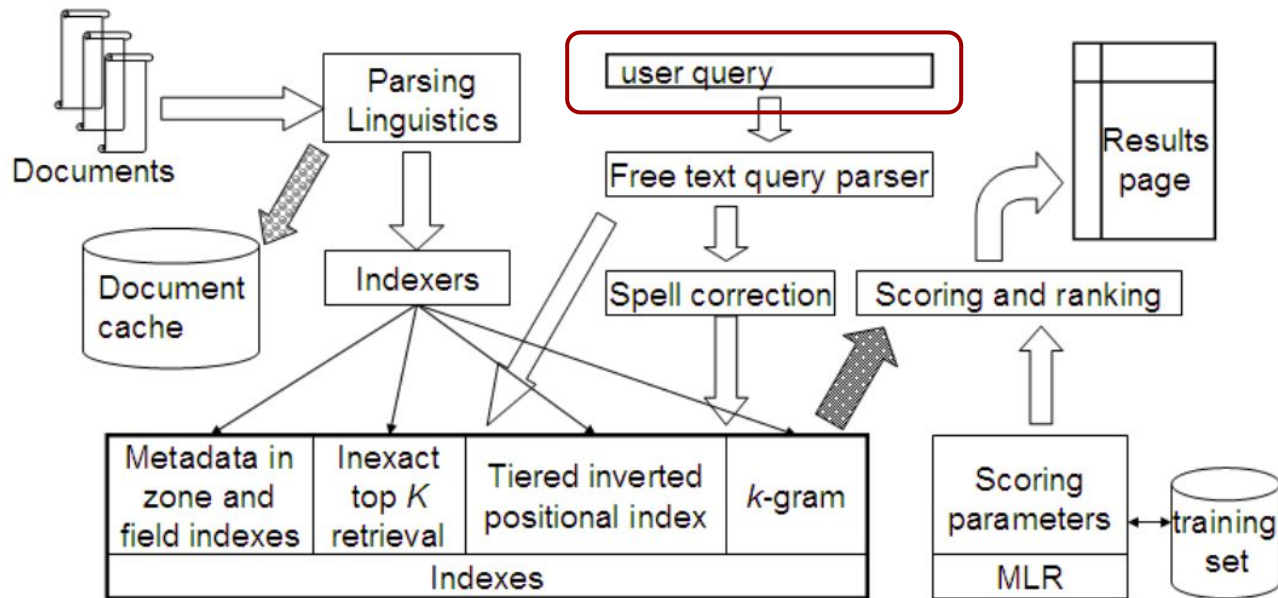
Putting them all together



Several Indexes are built:

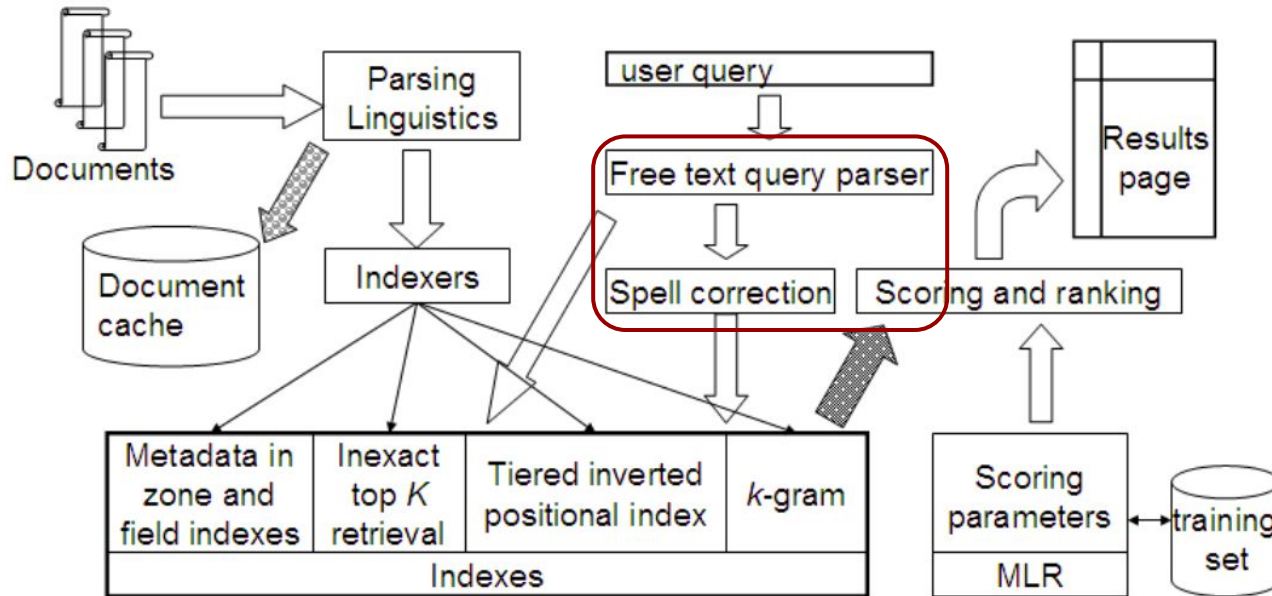
1. Zone Index
2. Field Index
3. Tiered Inverted Positional Index
4. Structures for accelerating inexact top-K retrieval
5. K-gram index

Putting them all together



User writes the query according to his information need

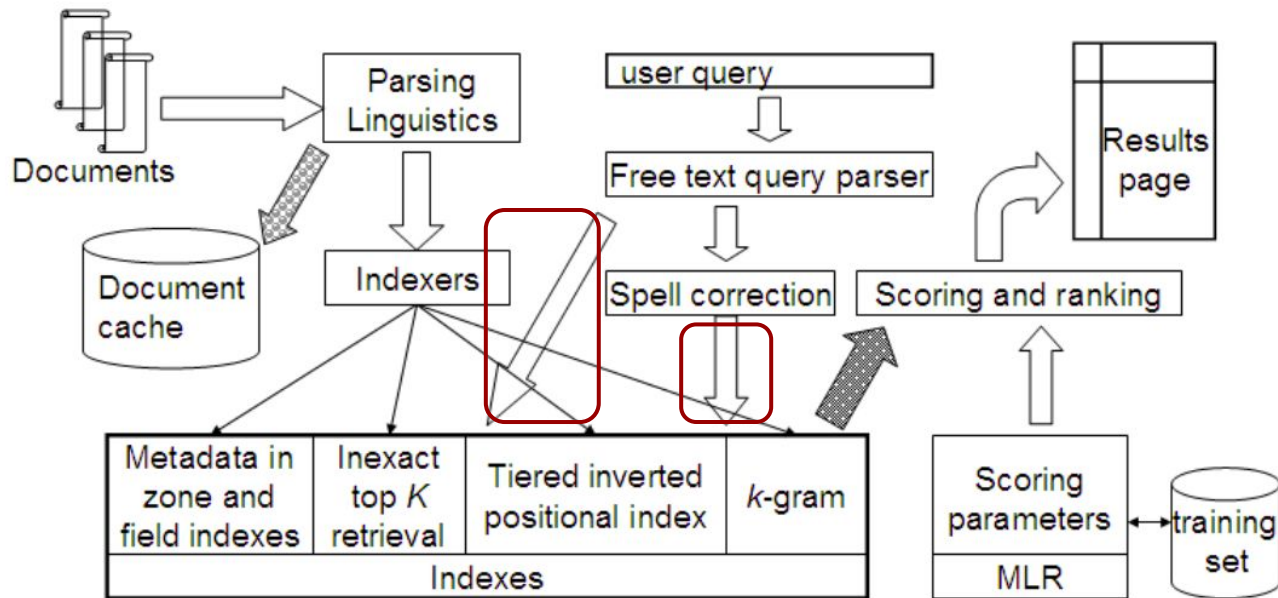
Putting them all together



The query is parsed,
tokens are extracted;

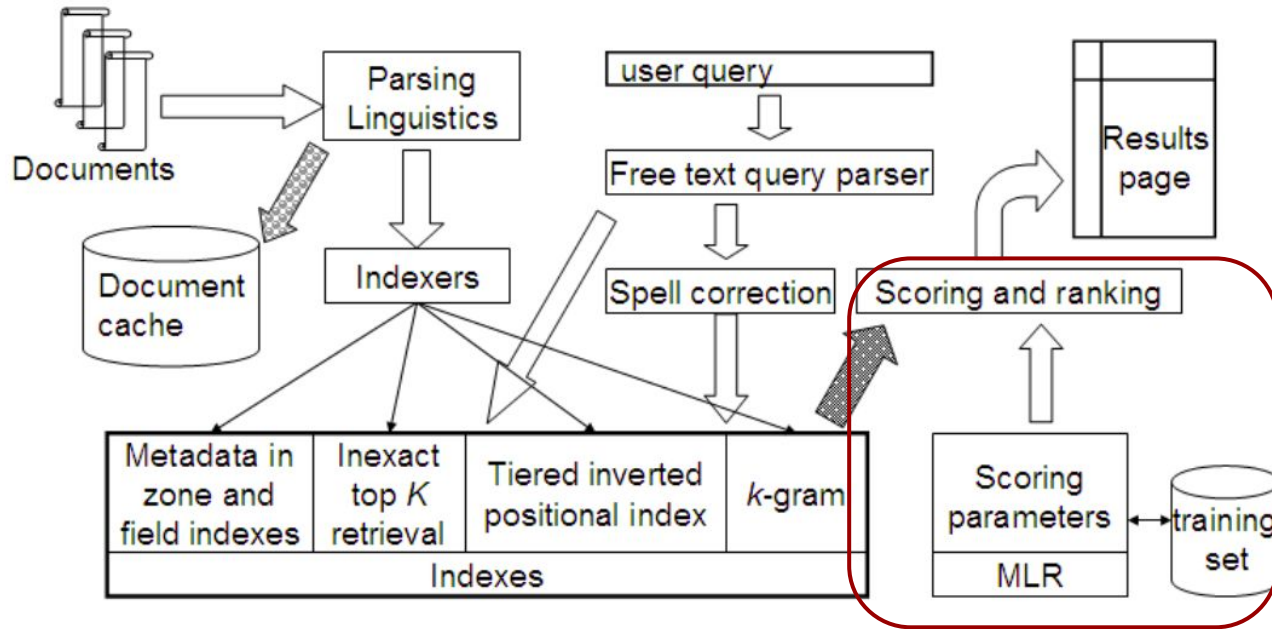
The Spell Correction
module may
optionally be invoked
only when the
original query fails
to retrieve enough
results

Putting them all together



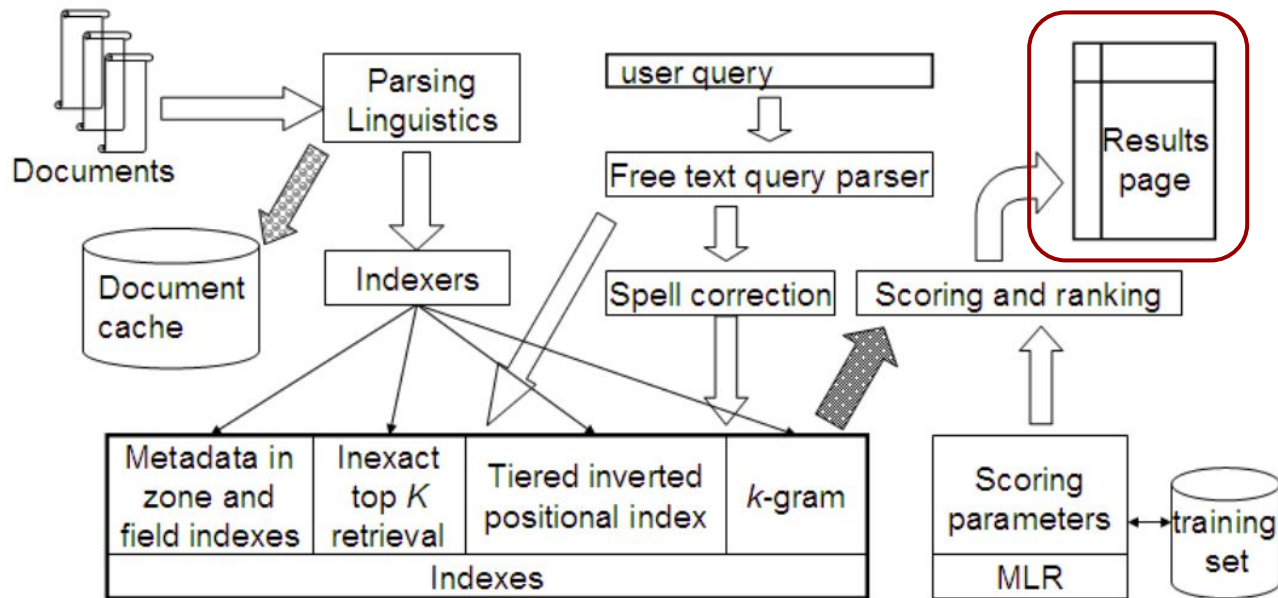
The query (or the corrected query) is used to retrieve documents

Putting them all together



Using the weight of the zones and fields, documents are scored and ranked. Zone weights are assigned using ML techniques

Putting them all together



Finally, these ranked documents are rendered as a results page